Descriptive Written Test – Assessment Tool 3

Course: Artificial Intelligence (CSA17)

Question 1: Intelligent Agent Types in a Smart Home Automation System

A smart home system must manage lighting, temperature, and security while responding to real-time inputs (motion, temperature change, time of day) and learning user preferences over time. Each of the four classic agent types would handle this task very differently.

Simple Reflex Agent

A simple reflex agent acts purely on the current percept, using condition–action ('if–then') rules with no memory of the past and no model of the world. In this system it would work like: *"IF motion detected THEN turn on light,"* or *"IF temperature > 28°C THEN turn on the AC."* This is fast and simple to implement, but it cannot distinguish context – for example, it cannot tell whether motion at 2 a.m. is the homeowner or an intruder, and it cannot remember that a room was already lit a moment ago. It cannot learn or adapt to user preference over time.

Model-Based Reflex Agent

A model-based agent maintains an internal state that tracks aspects of the world it cannot directly observe right now – for example, whether each room is currently occupied, what the last few motion events were, or what time of day/season it is. This lets the system reason more sensibly: it can keep a light on for a few minutes after motion stops (because its internal state says the room was recently occupied), or avoid triggering a security alert for a known, recurring motion pattern (like a pet). This is a clear improvement over the simple reflex agent because decisions depend on history and context, not just the instantaneous percept.

Goal-Based Agent

A goal-based agent selects actions by reasoning about which action sequence will achieve an explicit goal, rather than just reacting. For the smart home, a goal could be *"maintain room temperature between 22–24°C by 6 p.m."* or *"ensure all entry points are secured before the house is marked 'away'."* The agent can plan ahead – e.g., pre-cooling a room before the homeowner is expected to arrive, based on the goal and a predicted arrival time – instead of only reacting once the temperature is already uncomfortable.

Utility-Based Agent

A utility-based agent goes further by assigning a numeric utility (desirability) score to different possible states, so it can choose among several ways of satisfying a goal by picking the one with the highest overall value. This matters when goals compete: keeping the house very cool is good for comfort but costs more energy; keeping every light on is convenient but wastes power. A utility function combining comfort, energy cost, and security lets the agent make trade-offs – for example,

choosing a slightly less aggressive cooling setting that is 90% as comfortable but uses 40% less energy, or deciding whether a marginal motion event is worth a full security alert versus a quieter log entry.

Which type (or hybrid) is most effective?

A pure simple reflex agent is inadequate for this system because it cannot handle context, memory, or competing objectives. The most effective design is a **hybrid model-based, goal-based, and utility-based agent**:

- The **model-based** component maintains ongoing state – occupancy history, recent temperature trends, learned routines – so the system understands context rather than reacting blindly.
- The **goal-based** component drives planning toward explicit objectives, such as reaching a target temperature by a certain time or ensuring the house is fully secured before ‘away’ mode.
- The **utility-based** component resolves conflicts between comfort, energy efficiency, and security by scoring alternative actions and picking the one with the best overall trade-off, and it is also the natural place to incorporate learned user preferences over time (e.g., learning that this household prefers slightly warmer evenings but strict night-time security).

This hybrid approach directly matches the three stated priorities of the system – comfort, energy efficiency, and security – none of which can be fully achieved by any single agent type on its own.

Question 2: Uninformed Search for an Unknown Maze

A robot placed in an unknown maze, with no prior knowledge of the layout, must explore and find a path to the exit. Since no map is available beforehand, this is naturally addressed using uninformed (blind) search strategies, which explore the state space using only the problem definition (not a heuristic).

Uniform Cost Search (UCS)

UCS expands the frontier node with the lowest cumulative path cost g(n) so far, using a priority queue. Applied to the maze, if every move (e.g., one cell forward) has an equal cost, UCS behaves like Breadth-First Search, always finding the shortest path in terms of number of moves. If different moves have different costs (e.g., moving through a narrow, slow passage costs more than an open corridor), UCS still guarantees the cheapest path to the exit, since it always expands the least-cost node first and stops only when the goal is popped from the frontier.

Iterative Deepening Search (IDS)

IDS repeatedly runs Depth-Limited Search with increasing depth limits (0, 1, 2, …) until the exit is found. In the maze, the robot would first try all paths of length 0, then all paths up to length 1, then up to length 2, and so on. This combines the low memory usage of Depth-First Search with the completeness and optimality (for unit-cost problems) of Breadth-First Search, at the cost of re-exploring shallow parts of the maze multiple times.

Advantages and disadvantages (time and space complexity)

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
Uniform Cost Search (UCS)	O(b^(1+■C*/ε■)) where C* is the optimal cost and ε the minimum edge cost	Same as time – must keep the entire frontier and explored set in memory	Always finds the optimal (least-cost) path; complete if costs are positive	Can require large memory for big or deep mazes; slow if many equal-or-near-cost paths exist
Iterative Deepening Search (IDS)	O(b^d) – comparable to BFS/DFS asymptotically, with repeated shallow re-expansion overhead	O(b×d) – only needs to store the current path (like DFS), far less than UCS or BFS	Very low memory use; complete and optimal for unit-cost problems; good when maze depth is unknown	Repeats work at shallow depths on every iteration; not cost-aware unless combined with cost checks

Relating the problem to intelligent agent types

The maze-exploring robot is fundamentally a **goal-based agent**: its objective is explicitly to reach the exit, and it must choose a sequence of actions (moves) that achieves this goal rather than just reacting to the current cell. Because the maze layout is unknown in advance, the robot also needs a **model-based** component – it must maintain an internal map of cells it has already visited so it does not repeatedly re-explore dead ends, effectively building the search graph as it goes (similar to an online search agent). A purely reflex-based robot (reacting only to what is immediately visible with no memory) would be poorly suited here, since it could loop indefinitely in a maze without ever recognizing previously visited cells.

Agent design directly affects decision-making quality: a model-based, goal-based agent can systematically guarantee it eventually finds the exit (given a finite maze) and avoid repeating the same paths, whereas a simpler reflex design has no such guarantee, since it cannot represent or reason about the goal or its own search history.

Best combination and justification

The most effective combination is a **model-based, goal-based agent using Iterative Deepening Search** as its primary strategy, with Uniform Cost Search reserved for cases where move costs are genuinely unequal:

- If all moves cost the same (a typical grid maze), **IDS** is preferable: it finds the shortest path (same as BFS/UCS in this case) while using far less memory, O(b×d) instead of the exponential memory UCS/BFS would need – important since the robot has limited onboard memory and the maze size/depth is unknown in advance.
- If different passages have different traversal costs (e.g., rough terrain, narrow corridors, or risk zones), **UCS** becomes necessary, since IDS by itself does not account for cost, only depth.

In both cases, pairing the search strategy with a **model-based, goal-based agent** is essential: the model-based memory of visited cells prevents infinite loops and redundant exploration, and the goal-based reasoning ensures every expanded action is directed toward reaching the exit rather than moving arbitrarily. This combination balances completeness, optimality, and the practical memory constraints of a real robot exploring an unknown maze.